

ASEN 5264 - DMU

Homework #2

Bijan Jourabchi

Problem 1:

- a) R (reward function) maps each state and action to a reward. In other words, for a given state S and action A , the reward function tells what reward we will get from taking that action. The state-action value function (Q) represents the expected return when starting in state S and taking action A . In other words, Q can quantify our expected reward if we were to continue with greedy policies.

b)

$$Q(S, A) = R(S, A) + \gamma \sum_{S'} T(S' | S, A) V(S')$$

Problem 2:

- a)
- $S = [1, 2, 3, 4]$

 $A = [\text{roll}, \text{step}]$

$$T^{\text{roll}} = \begin{bmatrix} 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 0.5 & 0.5 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T^{\text{step}} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{cases} -1 & \text{if } S = 2, A = \text{step} \\ +3 & \text{if } S = 3, A = \text{step} \\ 0 & \text{otherwise} \end{cases}$$

- b) Given
- $Q^*(s, a) = R(s, a) + \gamma \sum T(S' | S, A) V(S')$

Since $S = 4$ is a terminal state, we can begin the iterative process at $S = 4$ and work down, giving us the following:

S	A	Q^*	V^*
4	N/A	0	0
3	Step	3	3
3	Roll	0	
2	Step	1.825	1.85
2	Roll	1.424	
1	Step	1.7575	2.30375
1	Roll	2.30375	

Thus, the optimal policy for this game is as follows:

$$\pi^* = \begin{cases} \text{Roll} & \text{if } S = 1 \\ \text{Step} & \text{if } S = 2 \\ \text{Step} & \text{if } S = 3 \end{cases}$$

Problem 3:

a) $S = [L, M, R]$

$A = [\text{roll}, \text{reset}]$

For the transition probability, let p be the probability the dice lands on an odd number, thus $P(\text{even}) = 1 - p$.

$$T^{\text{roll}} = \begin{bmatrix} 0 & p & 1-p \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$T^{\text{reset}} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

$$R = \begin{cases} +2 & \text{if } S = M, A = \text{reset} \\ -1 & \text{if } S = R, A = \text{reset} \\ 0 & \text{otherwise} \end{cases}$$

b) To evaluate the given policy, we can use the Bellman Expectation equation:

$$U^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} T(s' | s, \pi(s)) U^\pi(s')$$

Or, in matrix-vector form as:

$$\vec{U}^\pi = (I - \gamma T^\pi)^{-1} \vec{R}^\pi$$

Where I is the $s \times s$ identity matrix, T^π is the transition probabilities using the given policy at a state s . Solving for $U^\pi(s)$, letting $p = 0.5$ for a fair dice:

$$\vec{U}^\pi = \left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - 0.95 \begin{bmatrix} 0 & 0.5 & 0.5 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \right)^{-1} \begin{bmatrix} 0 \\ 2 \\ -1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & -0.475 & -0.475 \\ -0.95 & 1 & 0 \\ -0.95 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 2 \\ -1 \end{bmatrix}$$

$$\boxed{\vec{U}^\pi = \begin{bmatrix} 4.8718 \\ 6.6282 \\ 3.6282 \end{bmatrix}}$$

- c) For any given policy, if the reward can be bounded, $r_{\min} \leq r \leq r_{\max}$, then we can give finite upper and lower bounds for the expected discount score we can receive with the following formula:

$$\frac{r_{\min}}{1-\gamma} \leq \sum_{t=0}^{\infty} \gamma^t r_t \leq \frac{r_{\max}}{1-\gamma}$$

From our reward function, we see that $r_{\min} = -1, r_{\max} = 2$. Subbing this into our expression we get the following upper and lower bounds with $\gamma = 0.95$:

$$-20 \leq \sum_{t=0}^{\infty} \gamma^t r_t \leq 40$$

Problem 4:

Code for problem's 4 and 5 are below:

```

1 import Cairo
2 import Fontconfig
3
4 using DMUStudent.HW2
5 using POMDPs: states, actions, discount
6 using POMDPTools: ordered_states, render
7
8 #####
9 # Question 3
10 #####
11
12
13 function value_iteration(m)
14
15     # Problem Setup
16     A = actions(m)
17     T = transition_matrices(m, sparse = true)
18     R = reward_vectors(m)
19     S = states(m)
20     gamma = discount(m)
21     tol = 1e-6
22     n_states = length(S)
23     n_actions = length(A)
24
25     # Value fxn initialization
26     V = ones(n_states)
27     V_prev = zeros(n_states)
28
29     # State Value fxn
30     Q = zeros(n_states, n_actions)
31
32     while maximum(abs.(V-V_prev)) > tol
33
34         V = copy(V_prev)
35
36         for (i,a) in enumerate(A)

```

```
37         # R -> [S, 1], T-> [S x S], V -> [S x 1]
38         Q[:, i] = R[a] + gamma * (T[a] * V)
39     end
40
41     V_prev = vec(maximum(Q, dims=2)) # Col wise max
42
43     end
44
45     return V
46 end
47
48 @show actions(grid_world)
49 V = value_iteration(grid_world)
50 render(grid_world, color = V)
51
52 #for V in V_hist
53 #    render(grid_world, color = V)
54 #end
55
56 #####
57 # Question 4
58 #####
59
60 m = UnresponsiveACASMDP(20)
61
62
63 V = value_iteration(m)
64 @show HW2.evaluate(V, "bijan.jourabchi@colorado.edu")
```